POO

DEFINICIÓN

- ¿Qué significa Orientación a Objetos?
  - Organización del software como una colección de objetos que contienen tanto estructura como comportamiento.

DEFINICIÓN

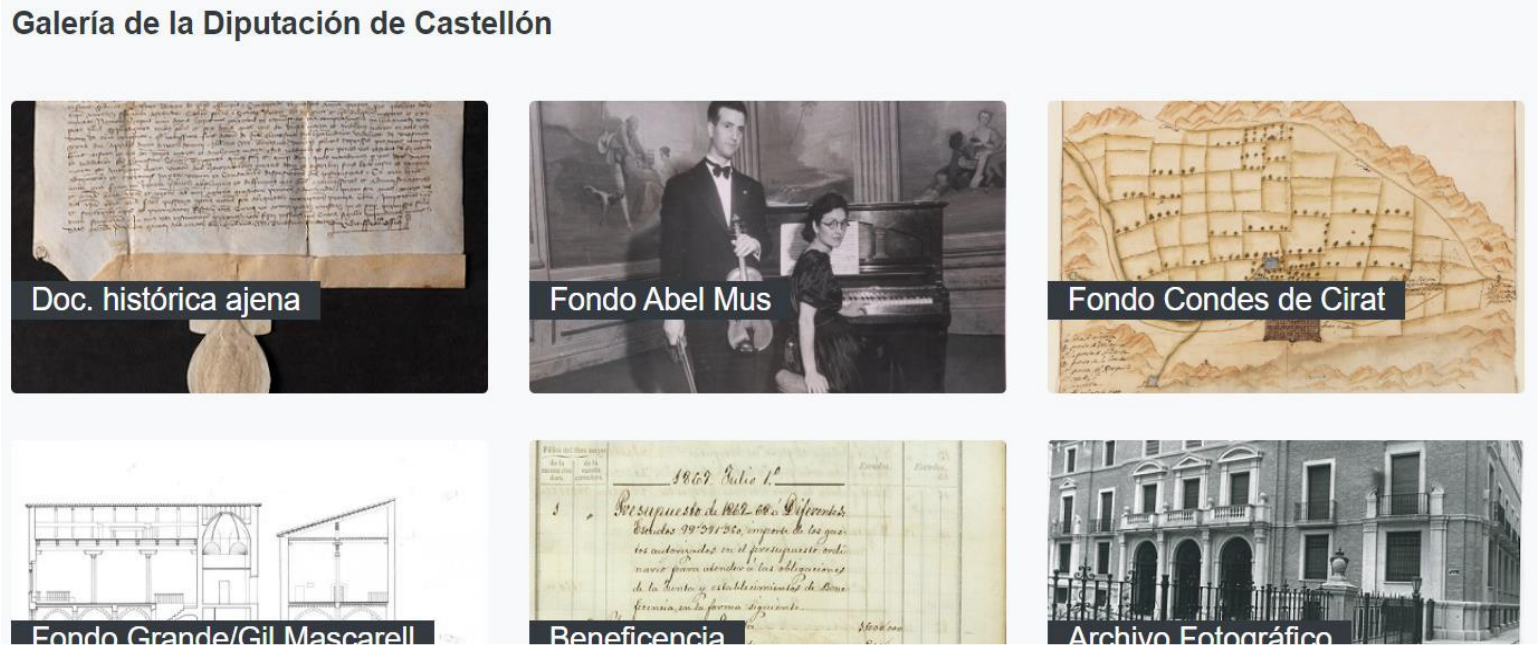
- Objetivos

- **Factores Externos**
  - Corrección
  - Robustez
  - Extensibilidad
  - Reutilización
  - Compatibilidad
  - Eficiencia
  - Portabilidad
  - Facilidad de uso
  - Funcionalidad
  - Oportunidad
- **Factores Internos**
  - Modularidad
  - Legibilidad

REUTILIZACIÓN

- Ejemplo
  - Nos piden realizar un proceso que a partir de una imagen en el sistema, calcule una imagen reducida y la guarde en base de datos, para mostrarla en la primera página del portal y no mostrar imágenes.

REUTILIZACIÓN



REUTILIZACIÓN

- Pseudocódigo
  1. Descargar imagen
  2. Comprobar que no exista ya una miniatura
  3. Calcular Thumbnail (miniatura)
  4. Asociar y guardar en base de datos

REUTILIZACIÓN

- Pasa un mes y resulta que necesitamos lo mismo pero para la primera imagen de un PDF otro proceso:
  1. Descarga PDF
  2. Comprobar que no exista
  3. Calcular Thumbnail (primera página)
  4. Asociar y guardar en base de datos

REUTILIZACIÓN

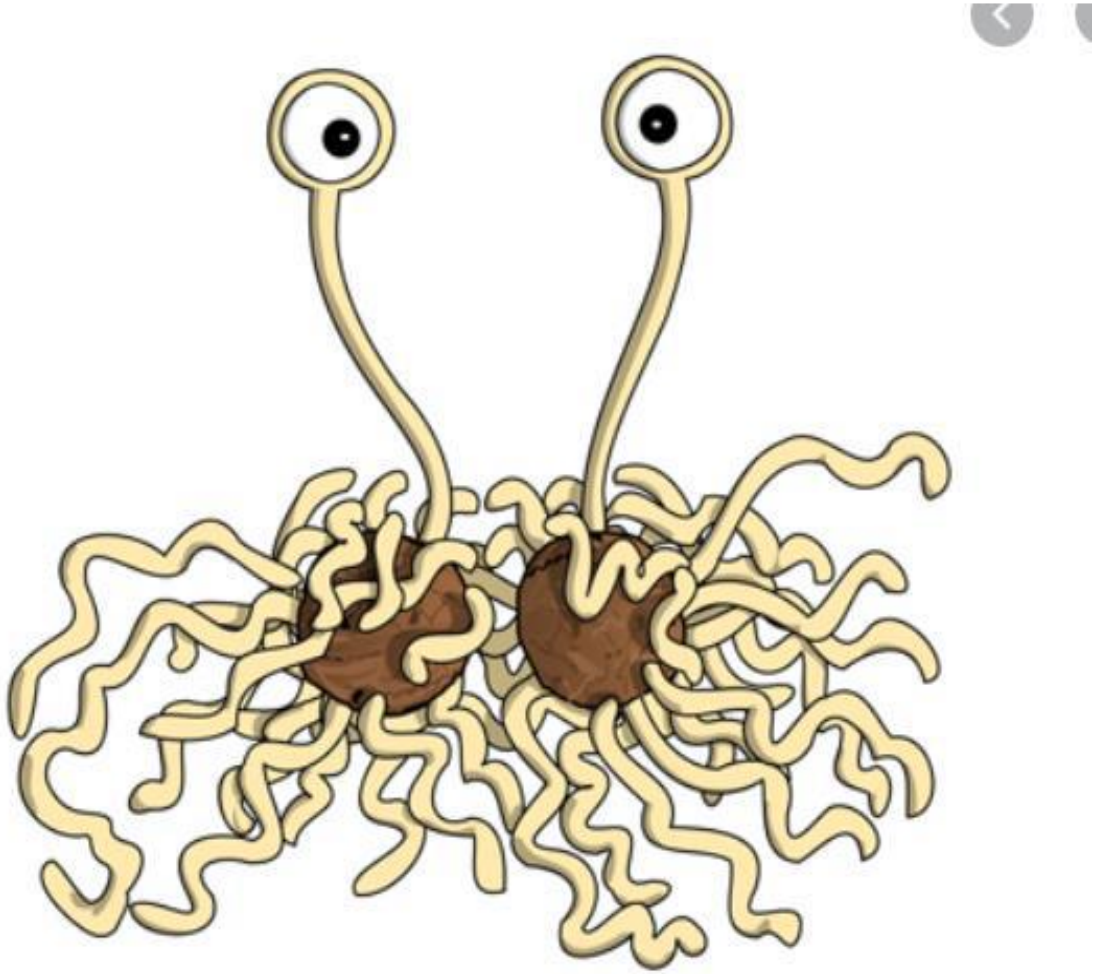
- De verdad se va a duplicar todo ese código ?
- Primera aproximación hacer funciones / módulos para reutilizar

REUTILIZACIÓN

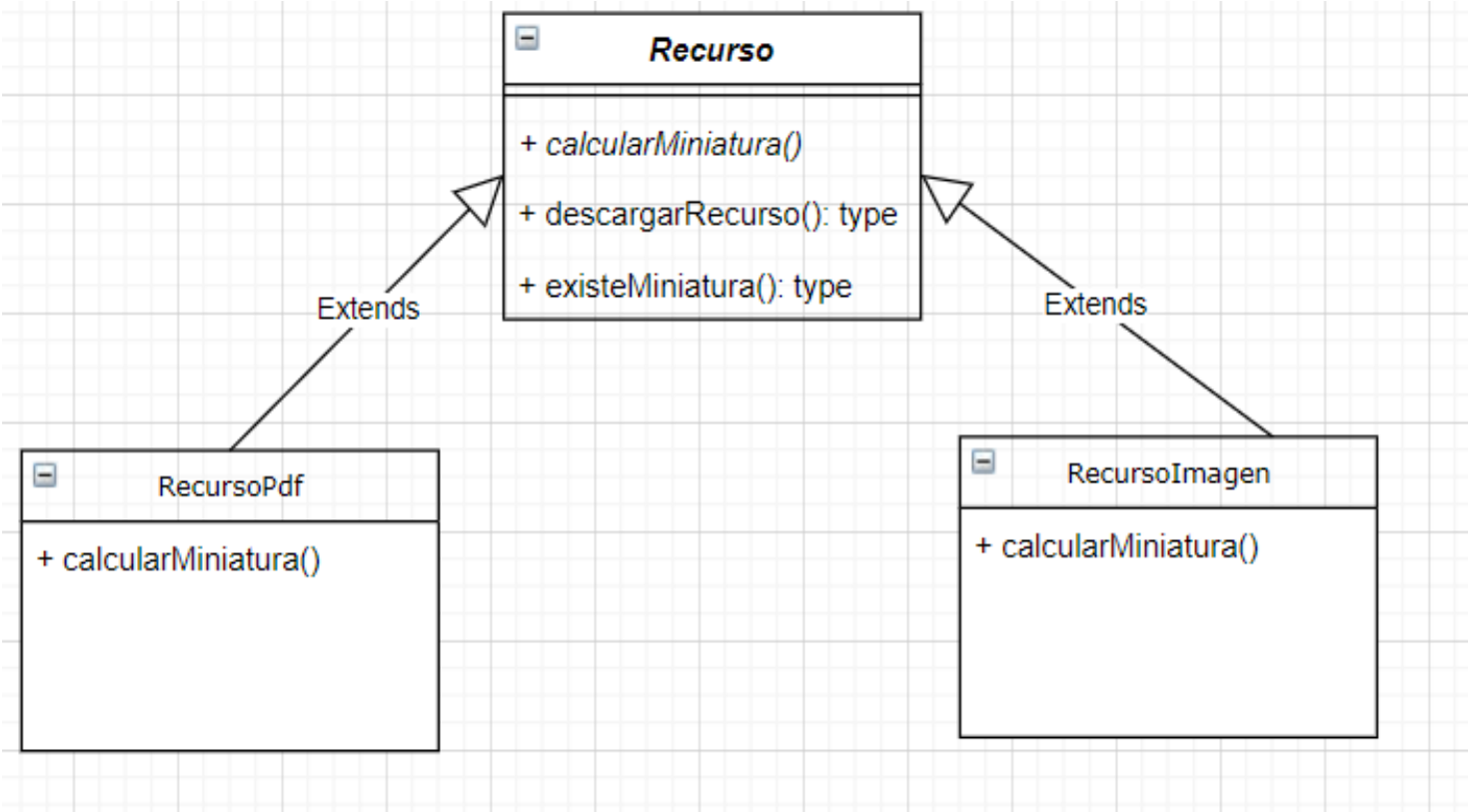
- Función1: descargarObjeto
- Función2: comprobarSiExisteMiniatura
- Función 3: calcular imagen
  - if (fichero is pdf) calcular miniatura primera página
  - else if (fichero is imagen) calcular miniatura imagen
- Función: 4 asociar

REUTILIZACIÓN

- Problema función 3 : spaguetti code
- if and if and if and if anidados dependiendo de cada uno de los casos



SOLUCIÓN: TIPADO Y HERENCIA

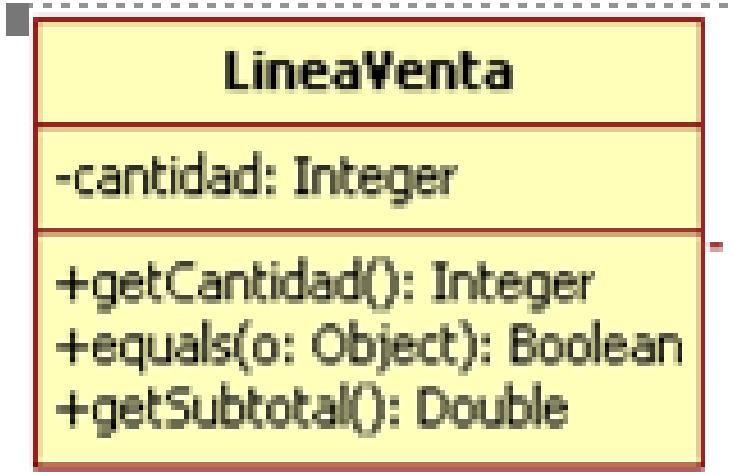


ALARMAS

- ¿Cuándo sé que hace falta OO / reutilización y nadie lo dice?
  - Cuando al hacer una corrección sobre algo que no funcionaba: “se nos olvidó hacerlo para x”
  - “Hay que acordarse de hacerlo para cuando el usuario es” o “cuando se entra como” o “ para el sistema”
  - “Hay que acordarse de probarlo para”

JAVA - CLASES

- Java es fuertemente tipado
- Implementación total o parcial de un TAD
- Componente de una clase
  - Atributos
  - Métodos



JAVA - OCULTACIÓN INFORMACIÓN

- Ocultación de la información
  - Public
  - Private
  - Protected
  - Package

JAVA - OBJETOS

- Objetos

```
Cuenta cta = new Cuenta (persona, 300.0);
```

- Es una instancia de una clase
- Cta sería un objeto de la clase Cuenta, cuando se ejecute esa línea de código
- New crea nuevos objetos

JAVA - CLASE ABSTRACTA

- Toda clase que contenga algún método abstracto
- método abstracto es uno que no tiene implementación y se espera que los que heredan lo implementen

```
public abstract class UserAuthCreationServiceImpl implements UserAutomaticCreationAuthSer

    @Autowired
    UsuariosGenericosService usuariosGenericosService;
    @Autowired
    PermissionControllerService permissionControllerService;

    abstract void checkAndCreateGroup(String userName, List<String> groupsInA3W);
```

JAVA - INTERFACES

- Solo contiene definiciones de métodos y constantes
- Los métodos son abstractos, no es necesario especificarlo
- Es una especificación

```java
public interface UserService {

    String getUserNameAsOpacTokenUser(User user);
    boolean isInteresado(User user, String id);
    boolean isOpacTokenUser(User user, String id);
    boolean isTramitador(User user);

}
```

```java
@Service(value = "userService")
public class UserServiceImpl implements UserService {
```

JAVA - CLASES GENÉRICAS

- Aseguran que los métodos se aplicarán (tendrán parámetros un cierto tipo declarado en tiempo de ejecución)

```
public void apilar(E valor) {
```

```
public class Pila <E> {
    private final int tamaño;
    private int superior;
    private E[] elementos;

    public Pila() {
        this (10); // tamaño predeterminado de la pila
    }
```